

```
# --- 1. SETUP & INSTALLATION ---
# Install the Google Generative AI SDK
!pip install -U -q google-generativeai

import os
import base64
import google.generativeai as genai
from getpass import getpass
from IPython.display import display, Markdown

# --- 2. AUTHENTICATION ---
# Securely handle your API key
api_key = getpass('Enter your Gemini API Key: ')
genai.configure(api_key=api_key)
```

Enter your Gemini API Key:

```
# --- 3. THE MINIMALIST LOGIC ---
def process_cv(pdf_path, job_description):
    """
    Processes a PDF CV against a JD using a minimalist recruiter persona.
    Logic mirrors the provided TypeScript implementation.
    """

    # Encode PDF to Base64 to pass as inline data
    with open(pdf_path, "rb") as f:
        pdf_base64 = base64.b64encode(f.read()).decode("utf-8")

    # Strict System Instruction to enforce the minimalist UI/UX
    system_instruction = """
    You are a Minimalist Technical Recruiter. Your task is to process a CV (PDF) against a provided Job Description.

    STRICT OUTPUT RULES:
    - Use bullet points only for lists.
    - No introductory or concluding remarks.
    - Be blunt and objective.
    - Follow the exact REPORT STRUCTURE provided below.

    REPORT STRUCTURE:
    ### 👤 Candidate: [Name]
    - **Experience:** [Total years]
    - **Top Skills:** [List top 5]

    ### 📏 Alignment
    - **Match Score:** [X/100]
    - **Verdict:** [Strong Hire / Potential / Reject]

    ### ❌ Gaps
    - [List specific missing requirements]

    ### 📌 To-Do
    - [3 short career tips]
    """

    # Initialize the model (Gemini 1.5 Flash is recommended for speed/cost)
    model = genai.GenerativeModel(
        model_name="gemini-2.5-flash",
        system_instruction=system_instruction
    )

    # Generate Content
    response = model.generate_content([
        {
            "mime_type": "application/pdf",
            "data": pdf_base64
        },
        f"Job Description:\n{job_description}"
    ], generation_config={"temperature": 0.1})

    return response.text
```

```
# --- 4. EXECUTION ---
# Define your inputs here
# Ensure "Ideal_Python_Developer_CV.pdf" is uploaded to your notebook environment[cite: 1, 2]
CV_FILENAME = "/jordan_CV.pdf"
JD_TEXT = """
Looking for a Senior Python Developer with 5+ years experience.
```

```
Required: Gemini API, FastAPI, Docker, and Kubernetes.
"""
```

```
print(CV_FILENAME)
```

```
/jordan_CV.pdf
```

```
if os.path.exists(CV_FILENAME):
    print(f"⌚ Analyzing {CV_FILENAME}...")
    report = process_cv(CV_FILENAME, JD_TEXT)
    display(Markdown("---"))
    display(Markdown(report))
else:
    print(f"❌ Error: {CV_FILENAME} not found. Please upload the PDF to the notebook files.")
```

⌚ Analyzing /jordan_CV.pdf...

 Candidate: Jordan Reed

- **Experience:** 5 years
- **Top Skills:** Python, Gemini API, FastAPI, Docker, Kubernetes

 Alignment

- **Match Score:** 100/100
- **Verdict:** Strong Hire

 Gaps

- None

 To-Do

- Quantify all achievements with metrics.
- Emphasize leadership roles more.
- Tailor project descriptions to specific job requirements.

```
# --- 4. EXECUTION ---
# Define your inputs here
# Ensure "Ideal_Python_Developer_CV.pdf" is uploaded to your notebook environment[cite: 1, 2]
CV_FILENAME = "/alex_CV.pdf"
JD_TEXT = """
Looking for a Senior Python Developer with 5+ years experience.
Required: Gemini API, FastAPI, Docker, and Kubernetes.
"""
```

```
if os.path.exists(CV_FILENAME):
    print(f"⌚ Analyzing {CV_FILENAME}...")
    report = process_cv(CV_FILENAME, JD_TEXT)
    display(Markdown("---"))
    display(Markdown(report))
else:
    print(f"❌ Error: {CV_FILENAME} not found. Please upload the PDF to the notebook files.")
```

⌚ Analyzing /alex_CV.pdf...

 Candidate: Alex Rivera

- **Experience:** 0.25 years
- **Top Skills:** Python, React, Docker, Node.js, SQL

 Alignment

- **Match Score:** 20/100
- **Verdict:** Reject

 Gaps

- 5+ years of experience
- Gemini API experience
- FastAPI experience
- Kubernetes experience

 To-Do

- Gain significant professional experience.
- Learn FastAPI and Kubernetes.
- Explore AI/ML APIs like Gemini.

Start coding or [generate](#) with AI.

